

УДК Lorem ipsum dolor sit amet

Операционная семантика выражений в языке Go на языке ABML

Харьков А.А. (Студент ММФ НГУ)

Ануреев И.С. (Институт систем информатики СО РАН)

В данной статье представлено формальное описание операционной семантики выражений языка программирования Go с использованием языка спецификаций ABML. Рассматривается фрагмент языка Go, не включающий в себя работу с многопоточностью и соответствующими конструкциями (горутины, каналы, операторы `go` и `select`). Предложенная онтологическая модель позволяет строго задать семантику базовых конструкций языка и может служить основой для разработки инструментов верификации программ.

Ключевые слова: операционная семантика, язык Go, язык ABML, формальные методы, онтологическое моделирование.

1. Введение

Формальное описание семантики языков программирования является фундаментальной задачей, лежащей в основе разработки компиляторов, статических анализаторов и инструментов верификации программ. Традиционные подходы к спецификации семантики, основанные на естественном языке, часто страдают от неоднозначности и неполноты, что затрудняет создание формально верифицируемых инструментов.

В данной работе рассматривается фрагмент языка программирования Go и его операционная семантика, формализованная с помощью языка спецификаций ABML (Attribute-Based Modeling Language), который является лексическим расширением диалекта Common Lisp — SBCL. Выбор языка Go обусловлен его возрастающей популярностью в промышленной разработке, особенно в области системного программирования. Язык ABML, в свою очередь, предоставляет средства для описания онтологий предметных областей и формализации правил перехода между состояниями.

2. Онтология фрагмента языка Go

Построение формальной модели семантики языка программирования требует предварительного создания онтологии — системы понятий, описывающих синтаксические конструкции и семантические сущности языка. В данном разделе на языке ABML вводятся типы объектов, соответствующие элементам фрагмента языка Go.

Классификация и порядок изложения опираются на официальную спецификацию языка Go. Большинство наименований моделируемых объектов соответствуют их названиям в спецификации.

2.1. Имена

Все имена в программе на Go моделируются как строки. Вводятся следующие типы имён:

```
(typedef "identifier" (uniont string))  
(typedef "constant name" "identifier")  
(typedef "variable name" "identifier")  
(typedef "field name" "identifier")  
(typedef "function name" "identifier")  
(typedef "parameter name" "identifier")  
(typedef "method name" "identifier")  
(typedef "label name" "identifier")  
(typedef "type name" "identifier")
```

2.2. Типы данных

Система типов языка Go разделяется на базовые (простые) и составные типы.

```
(typedef "type" (uniont "primitive type" "composite type"))
```

2.2.1. Базовые типы

Базовые типы включают логический, числовые и строковый.

```
(typedef "primitive type" (uniont "bool type" "numeric type" "string type"))
(cot "bool type")
(typedef "numeric type" (uniont "int type" "float type" "complex type"))
(cot "string type")
```

Числовые типы детализируются. Целые типы разделяются на знаковые и беззнаковые с указанием разрядности. Вещественные и комплексные типы также имеют разрядность.

```
(typedef "int type" (uniont "signed int type" "unsigned int type"))
(cot "signed int type" :at "bit size" (enumt 8 16 32 64))
(cot "unsigned int type" :at "bit size" (enumt 8 16 32 64))
(cot "float type" :av "bit size" (enumt 32 64))
(cot "complex type" :av "bit size" (enumt 64 128))
```

2.2.2. Составные типы

Составные типы включают массивы, срезы, структуры, указатели, функции, методы, интерфейсы, карты и каналы.

```
(typedef "composite type" (uniont "array type" "slice type" "struct type"
                                "pointer type" "function type" "method type"
                                "interface type" "map type" "channel type"))
```

Массивы имеют фиксированную длину и тип элементов. Срезы — только тип элементов.

```
(cot "array type" :at "elem type" "type" :at "len" nat)
(cot "slice type" :at "elem type" "type")
```

Структура определяется набором полей. Атрибут `ordered` используется на этапе статического анализа для поддержки позиционной инициализации.

```
(cot "struct type" :at "fields" (cot :amap "field name" "type")
:at "ordered" (listt "field name"))
```

Указатель хранит тип, на который он указывает.

```
(cot "pointer type" :at "type" "type")
```

Функциональный тип описывает сигнатуру функции. Имена параметров опущены, так как при проверке типов важны только сами типы. Вариативный параметр (многоточие) моделируется отдельным атрибутом.

```
(cot "function type"
  :at "param types" (listt "type")
  :at "variadic type" "type"
  :at "result types" (listt "type"))
```

Тип метода включает получателя (receiver) и используется при определении интерфейсов.

```
(cot "method type"
  :at "receiver type" "type"
  :at "param types" (listt "type")
  :at "variadic type" "type"
  :at "result types" (listt "type"))
```

Интерфейс определяется как набор методов с их сигнатурами. Карта задаётся типом ключа и типом значения. Канал имеет тип передаваемых элементов; направление не моделируется, так как для корректных программ оно не влияет на семантику.

```
(cot "interface type" :at "methods" (cot :amap "method name" "method type"))
(cot "map type"      :at "key type" "type" :at "elem type" "type")
(cot "channel type"  :at "elem type" "type")
```

2.3. Ячейки памяти и значения

Моделирование памяти является основой операционной семантики. Ячейка представляет собой изменяемый контейнер, хранящий значение. Переменные связываются с ячейками, а не непосредственно со значениями. Тип ячейки указывает на тип хранимого значения.

```
(mot "cell" :at "value" "Go value" :at "type" "type")
(typedef "Go value" (uniont "untyped constant" "typed primitive" "composite
value"))
```

Нетипизированные константы существуют только на этапе статического анализа; в операционной семантике они преобразуются в типизированные значения.

```
(typedef "untyped constant" (uniont bool int real string complex))
(mot "typed primitive"
  :at "type" "type"
  :at "value" (uniont "nil" "untyped constant"))
(cot "nil")
```

Составные значения моделируют структуры данных языка Go. Массив хранит элементы в виде списка ячеек, что обеспечивает возможность изменения элементов.

```
(typedef "composite value" (uniont "array value" "slice value" "struct value"
                                "map value" "interface value" "method value"
                                "function value" "cell" "channel value"))

(mot "array value"
  :at "type"      "array type"
  :at "elements" (listt "cell"))
```

Срез является представлением фрагмента массива. Несколько срезов могут разделять один и тот же массив.

```
(mot "slice value"
  :at "type"      "slice type"
  :at "array"     "array value"
  :at "offset"    nat
  :at "length"    nat
  :at "capacity" nat)
```

Структура хранит поля как отображение имён на ячейки.

```
(mot "struct value"
  :at "type"    "struct type"
  :at "fields" (mot :amap "field name" "cell"))
```

Карта хранит записи как отображение ключей на ячейки значений.

```
(mot "map value"
  :at "type"    "map type"
  :at "entries" (mot :amap "Go value" "cell"))
```

Канал представляет собой FIFO-очередь с буфером и очередями ожидающих горутин (моделируемых ячейками).

```
(mot "channel value"
  :at "type"          "channel type"
  :at "buffer"        (listt "Go value")
  :at "send queue"    (listt "cell")
  :at "receive queue" (listt "cell")
  :at "closed"        bool)
```

Функция хранит тип, сигнатуру (с именами параметров), тело и замыкание — отображение захваченных переменных на ячейки.

```
(mot "function value"
  :at "type"          "function type"
  :at "signature"     "function signature"
  :at "body"          "block"
  :at "closure"       (mot :amap "variable name" "cell"))
```

Сигнатура функции содержит список деклараций параметров, вариативный параметр (если есть) и результаты (могут быть безымянными — только типы, или именованными).

```
(mot "function signature"
```

```

      :at "parameters"      (listt "param decl")
      :at "variadic param" "param decl"
      :at "results"        (uniont (listt "type") (listt "param decl")))

(mot "param decl" :at "type" "type" :at "name" "parameter name")

```

Метод аналогичен функции, но имеет отдельный получатель. Сигнатура метода включает получатель как отдельную декларацию параметра.

```

(mot "method value"
  :at "type"      "method type"
  :at "signature" "method signature"
  :at "body"      "block"
  :at "closure"   (mot :amap "variable name" "cell"))

(mot "method signature"
  :at "receiver"      "param decl"
  :at "parameters"    (listt "param decl")
  :at "variadic param" "param decl"
  :at "results"       (listt "param decl"))

```

Интерфейс хранит статический тип и значение. Динамический тип содержится внутри значения.

```

(mot "interface value"
  :at "type" "interface type"
  :at "value" "typed value")

```

2.4. Литералы

Литералы представляют собой значения, которые могут быть непосредственно записаны в исходном коде. Общий тип литерала объединяет типизированные константы и составные литералы.

```
(typedef "literal" (uniont "typed constant" "composite lit"
                        "function lit" "method lit"))

(typedef "composite lit" (uniont "array lit" "slice lit"
                                "struct lit" "map lit"))
```

Литералы массивов и срезов могут содержать явное указание индексов. Каждый элемент представлен парой `index & elem`.

```
(mot "index & elem" :at "index" nat :at "value" "expression")

(mot "array lit"
  :at "type"      "array type"
  :at "elements" (listt "index & elem"))

(mot "slice lit"
  :at "type"      "slice type"
  :at "elements" (listt "index & elem"))
```

Литералы структур могут использовать явные имена полей.

```
(mot "field & value" :at "field" "field name" :at "value" "expression")

(mot "struct lit"
  :at "type"      "struct type"
  :at "fields" (listt "field & value"))
```

Литералы карт задают пары ключ-значение.

```
(mot "key & elem" :at "key" "expression" :at "value" "expression")

(mot "map lit"
  :at "type"      "map type")
```



```
:at "elements" (listt "key & elem"))
```

Функциональные литералы (анонимные функции) хранят тип, сигнатуру и тело.

```
(mot "function lit"
  :at "type"      "function type"
  :at "signature" "function signature"
  :at "body"      "block")

(mot "method lit"
  :at "type"      "method type"
  :at "signature" "method signature"
  :at "body"      "block")
```

2.5. Выражения

Выражения в языке Go — это конструкции, которые после вычисления возвращают значение. Все выражения содержат атрибут `type`, значение которого вычисляется на этапе статического анализа.

```
(typedef "expression" (uniont "literal" "variable ref" "(1)" "conversion"
  "method expr" "selector expr" "index expr"
  "slice expr" "type assertion" "<−1"
  "function call" "unary expression"
  "binary expression"))
```

Ссылка на переменную представляет использование уже объявленной переменной.

```
(mot "variable ref" :at "name" "variable name" :at "type" "type")
```

Выражение в скобках влияет только на порядок вычисления.

```
(mot "(1)" :at 1 "expression" :at "type" "type")
```

Приведение типов позволяет явно преобразовывать значения.

```
(mot "conversion" :at "type" "type" :at "value" "expression")
```

Метод-выражение возвращает функцию с получателем в качестве первого параметра.

```
(mot "method expr" :at "receiver type" "type" :at "name" "method name"
      :at "type" "type")
```

Выбор поля или метода по экземпляру.

```
(mot "selector expr" :at "receiver" "expression" :at "name" "identifier"
      :at "type" "type")
```

Индексирование применимо к массивам, срезам, картам и строкам.

```
(mot "index expr" :at "indexable" "expression" :at "index" "expression"
      :at "type" "type")
```

Выражение среза создаёт новый срез из существующего массива, строки или среза. Параметры `low`, `high` и `max` задают границы; последний параметр может отсутствовать.

```
(mot "slice expr"
      :at "sequence" "expression"
      :at "low"      "expression"
      :at "high"     "expression"
      :at "max"      "expression"
      :at "type"     "slice type")
```

Проверка типа применяется к интерфейсным значениям.

```
(mot "type assertion" :at "interface" "expression" :at "type" "type")
```

Операция чтения из канала.

```
(mot "<=1" :at 1 "expression" :at "type" "type")
```

Вызов функции и метода.

```
(mot "function call" :at "function" "expression"
      :at "arguments" (listt "expression") :at "type" "type")

(mot "method call" :at "receiver" "expression" :at "method" "method name"
      :at "arguments" (listt "expression") :at "type" "type")
```

Унарные выражения используют цифровую нотацию для обозначения операнда.

```
(typedef "unary expression" (uniont "+1" "-1" "^1" "!1" "*1" "&1"))

(mot "+1" :at 1 "expression" :at "type" "type")
(mot "-1" :at 1 "expression" :at "type" "type")
(mot "^1" :at 1 "expression" :at "type" "type")
(mot "!1" :at 1 "expression" :at "type" "type")
(mot "*1" :at 1 "expression" :at "type" "type")
(mot "&1" :at 1 "expression" :at "type" "type")
```

Бинарные выражения также используют цифровую нотацию: **1** — левый операнд, **2** — правый.

```
(typedef "binary expression" (uniont "1||2" "1&&2" "1+2" "1-2" "1*2" "1/2"
      "1%2"
      "1<<2" "1>>2" "1&2" "1&^2" "1|2" "1^2"
      "1==2" "1!=2" "1<2" "1<=2" "1>2" "1>=2"))
```

2.6. Блоки

Блоки определяют области видимости переменных и меток. В языке Go блок представляет собой последовательность операторов, заключённую в фигурные скобки.

```
(mot "block"
      :at "statements" (listt "statement")
```

```

:at "label positions" (cot :amap "label name" nat)
:at "all variables"   (listt "variable name")
:at "decl variables"  (listt "variable name")
:at "variable cells"  (cot :amap "variable name" "cell"))

```

Атрибут `statements` содержит список операторов блока. Атрибут `label positions` сопоставляет меткам их позиции в блоке. Атрибут `variable cells` сохраняет для каждой переменной её ячейку памяти до входа в блок, что позволяет корректно обрабатывать затенение имён переменных.

2.7. Декларации

Декларации вводят новые объекты в программу.

```

(typedef "declaration" (uniont "type decl" "const decl" "var decl"
                                "function decl" "method decl"))

```

Декларация типа связывает имя с определением типа.

```

(mot "type decl" :at "name" "type name" :at "type" "type")

```

Декларация констант и переменных может содержать явное указание типов (опционально) и значения.

```

(mot "const decl"
  :at "names" (listt "constant name")
  :at "types" (listt "type")
  :at "values" (listt "expression"))

(mot "var decl"
  :at "names" (listt "variable name")
  :at "types" (listt "type")
  :at "values" (listt "expression"))

```

Декларация функций и методов.

```
(mot "function decl" :at "name" "function name" :at "value" "function lit")
(mot "method decl" :at "name" "method name" :at "value" "method lit")
```

2.8. Операторы

Операторы в языке Go представляют собой конструкции, которые изменяют состояние программы, но не возвращают значений.

```
(typedef "statement" (uniont "declaration" "label" "empty stmt" "1++" "1—"
    "assignment stmt" "if stmt" "switch stmt" "for
    stmt"
    "return stmt" "break stmt" "continue stmt" "goto
    stmt"
    "defer stmt" "go stmt" "1<-2" "fallthrough"
    "select stmt"))
```

Помеченный оператор позволяет присвоить метку любому оператору программы.

```
(mot "label" :at "name" "label name" :at "statement" "statement")
```

Пустой оператор не выполняет никаких действий.

```
(cot "empty stmt")
```

Операторы инкремента и декремента являются операторами, а не выражениями, что отличает Go от многих других языков.

```
(mot "1++ stmt" :at 1 "expression")
(mot "1— stmt" :at 1 "expression")
```

Операторы присваивания включают простое присваивание и составные присваивания с арифметическими и битовыми операциями.

```
(typedef "assignment stmt" (uniont "1=2" "1+=2" "1-=2" "1*=2" "1/=2" "1%=2"
```

```
"1<=>2" "1>=>2" "1^=2" "1|=2" "1&=2"
"1&^=2"))
```

Условный оператор может содержать необязательную инициализирующую часть.

```
(mot "if stmt"
  :at "init"      "statement"
  :at "condition" "expression"
  :at "then"      "block"
  :at "else"      (uniont "if stmt" "block"))
```

Оператор выбора существует в двух формах: переключатель по выражению и переключатель по типу.

```
(typedef "switch stmt" (uniont "expr switch stmt" "type switch stmt"))

(cot "default")
(cot "fallthrough")

(mot "expr switch stmt"
  :at "init"      "statement"
  :at "controlled" "expression"
  :at "cases"     (listt "expr case clause"))

(mot "expr case clause"
  :at "cases"     (uniont (listt "expression") "default")
  :at "statements" (listt "statement"))

(mot "type switch stmt"
  :at "init"      "statement"
  :at "name"      "variable name"
  :at "value"     "expression"
  :at "cases"     (listt "type case clause"))
```

```
(mot "type case clause"
  :at "cases"      (uniont (listt "type") "default")
  :at "statements" (listt "statement"))
```

Оператор цикла может быть представлен в нескольких формах: классический цикл с условием, цикл с инициализацией и пост-действием, а также циклы `range` для массивов, срезов, строк, карт, каналов и целых чисел (начиная с Go 1.22).

```
(typedef "for stmt" (uniont "for condition" "for clause"
                          "for range indexable" "for range map"
                          "for range channel" "for range int"))

(mot "for condition"
  :at "condition" "expression"
  :at "body"      "block")

(mot "for clause"
  :at "init"      "statement"
  :at "condition" "expression"
  :at "post"      "statement"
  :at "body"      "block")

(mot "for range indexable"
  :at "index"      "variable name"
  :at "value"      "variable name"
  :at "operation"  (enumt "assign" "decl")
  :at "indexable" "expression"
  :at "body"      "block")

(mot "for range map"
  :at "key"        "variable name")
```

```

      :at "value"      "variable name"
      :at "operation" (enumt "assign" "decl")
      :at "map"        "expression"
      :at "body"       "block")

(mot "for range channel"
  :at "value"      "variable name"
  :at "operation" (enumt "assign" "decl")
  :at "channel"    "expression"
  :at "body"       "block")

(mot "for range int"
  :at "value"      "variable name"
  :at "operation" (enumt "assign" "decl")
  :at "max"        "expression"
  :at "body"       "block")

```

Операторы передачи управления включают `return`, `break`, `continue` и `goto`. Оператор `defer` откладывает выполнение функции до возврата из окружающей функции. Оператор `go` запускает функцию в отдельной горутине.

```

(mot "return stmt"  :at 1 (listt "expression"))
(mot "break stmt"   :at 1 "label name")
(mot "continue stmt":at 1 "label name")
(mot "goto stmt"    :at 1 "label name")
(mot "defer stmt"   :at 1 "expression")
(mot "go stmt"      :at 1 "expression")

```

Оператор отправки в канал и оператор `select` для работы с несколькими каналами.

```

(mot "l<-r" :at 1 "expression" :at 2 "expression")

(mot "select stmt" :at "cases" (listt "common clause"))

```



```

(mot "common clause"
  :at "case"      (uniont "send stmt" "receive assign" "receive decl"
    "default")
  :at "statements" (listt "statement"))

(mot "receive assign" :at 1 (listt "expression") :at 2 "expression")
(mot "receive decl"  :at 1 (listt "variable name") :at 2 "expression")

```

3. Операционная семантика выражений

Операционная семантика определяет, как вычисляются выражения языка Go в процессе выполнения программы. В данном разделе на языке ABML формализуются правила вычисления для всех конструкций языка: литералов, выражений, деклараций, операторов.

Вычисление происходит в контексте окружения — агента, который хранит информацию о переменных, константах, функциях и типах. Каждый шаг вычисления представлен атрибутивным замыканием (`aclosure`), которое задаёт атрибут `opsem::rvalue` для выражений, возвращающих значения, `opsem::lvalue` для получения ячеек памяти, и `opsem` для операторов, не возвращающих значений.

3.1. Окружение и агенты

Окружение представляет собой совокупность агентов, каждый из которых моделирует состояние выполнения программы.

```

(mot "env" :at "agents" (listt "agent"))

(mot "agent"
  :at "constant value" (mot :amap "constant name" "Go value")
  :at "variable cell"  (mot :amap "variable name" "cell")
  :at "function value" (mot :amap "function name" "function value")
  :at "method value"   (mot :amap "type name"

```

```

                                (mot :amap "method name" "method value"))
:at "type"                      (mot :amap "type name" "type")
:at "value"                     "Go value")

```

3.2. Базовое правило для значений

Если выражение уже является значением, оно возвращается без изменений.

```
(aclosure ac :attribute "opsem::rvalue" :type "Go value" :instance i :do i)
```

3.3. Литерал массива

Вычисление литерала массива происходит в несколько этапов. Сначала вычисляется значение по умолчанию для элементов массива на основе типа элементов.

```

(aclosure ac :attribute "opsem::rvalue" :type "array lit" :instance i :stage
  nil :do
    (update-push-aclosure ac :stage "fill defaults")
    (clear-update-eval-aclosure ac :attribute "default value by type"
      :instance (aget i "type" "elem type")))

```

Затем создаётся массив из ячеек памяти, заполненных значениями по умолчанию.

```

(aclosure ac :attribute "opsem::rvalue" :type "array lit" :instance i :stage
  "fill defaults"
  :ap i "elements" es :ap i "type" at :ap at "elem type" et
  :ap (mo "pointer type" :av "type" et) ct :value v :do
    (match :v (null es) T
      :do (mo "array value" :av "type" at :av "elements" nil)
      :exit (update-push-aclosure ac :stage "evaluating" :av "left" es
        :av "array" (let ((c nil))
          (dotimes (k (aget at "len") c)
            (setf c (cons (mo "cell" :av "value" v :av "type" ct)

```

```

c))))))
(clear-update-eval-aclosure ac :instance (aget (car es) "value"))

```

На этапе вычисления явных элементов происходит замена значений в соответствующих ячейках.

```

(aclosure ac :attribute "opsem::rvalue" :type "array lit" :instance i :stage
  "evaluating"
  :ap ac "left" left :ap ac "array" arr :value v :ap (car left) "index" k :do
    (let ((c arr))
      (dotimes (j k (setf (car c) "value" v))
        (setf c (cdr c))))
    (match :v (null (cdr left)) T
      :do (mo "array value" :av "type" (aget i "type") :av "elements" arr)
      :exit (update-push-aclosure ac :av "left" (cdr left) :av "array" arr)
        (clear-update-eval-aclosure ac
          :instance (aget (car (cdr left)) "value"))))

```

3.4. Литерал среза

Вычисление литерала среза аналогично вычислению массива. Сначала вычисляется значение по умолчанию.

```

(aclosure ac :attribute "opsem::rvalue" :type "slice lit" :instance i :stage
  nil :do
    (update-push-aclosure ac :stage "build value")
    (update-push-aclosure ac :stage "fill defaults")
    (clear-update-eval-aclosure ac :attribute "default value by type"
      :instance (aget i "type" "elem type")))

```

По максимальному индексу создаётся массив нужного размера.

```

(aclosure ac :attribute "opsem::rvalue" :type "slice lit" :instance i :stage

```

```

"fill defaults"

:value v :ap i "elements" es :p -1 m

:ap (mo "pointer type" :av "type" (mo "pointer type"
    :av "type" (aget i "type" "elem type"))) ct :do
(dolist (e es) (let ((k (aget e "index"))) (if (> k m) (setf m k))))
(match :v (null es) T :do nil

:exit (update-push-aclosure ac :stage "evaluating" :av "left" es
    :av "array" (let ((c nil))
        (dotimes (k (1+ m) c)
            (setf c (cons (mo "cell" :av "value" v :av "type" ct)
                c)))))

(clear-update-eval-aclosure ac :instance (aget (car es)
"value"))))

```

После вычисления всех явных элементов возвращается их список в правильном порядке.

```

(aclosure ac :attribute "opsem::rvalue" :type "slice lit" :instance i :stage
    "evaluating"

:ap ac "left" left :ap ac "array" arr :value v :p (aget (car left)
    "index") k :do
(let ((c arr))
    (dotimes (j k (setf (car c) "value" v))
        (setf c (cdr c))))
(match :v (null left) T :do (reverse arr)

:exit (update-push-aclosure ac :av "left" (cdr left) :av "array" arr)
    (clear-update-eval-aclosure ac
        :instance (aget (car (cdr left)) "value"))))

```

На последнем этапе создаётся значение среза, ссылающееся на созданный массив.

```

(aclosure ac :attribute "opsem::rvalue" :type "slice lit" :instance i :stage
    "build value"

:value arr :ap i "type" stp :ap stp "elem type" etp :p (length arr) n :do

```

```

(mo "slice value"
  :av "type" (aget i "type")
  :av "array" (mo "array value"
    :av "type" (mo "array type" :av "elem type" etp :av "len" n)
    :av "elements" arr)
  :av "offset" 0
  :av "length" n
  :av "capacity" n))

```

3.5. Литерал структуры

Вычисление литерала структуры включает заполнение полей значениями по умолчанию.

```

(aclosure ac :attribute "opsem::rvalue" :type "struct lit" :instance i :stage
  nil
:p (aget i "type" "fields") tfs :p (attributes tfs) tns :do
  (update-push-aclosure ac :stage "eval field values")
  (match :v (null ns) T :do nil
    :exit (update-push-aclosure ac :stage "filling defaults"
      :av "type field names" tns
      :av "struct value" (mo :amap "field name" "cell"))
    (clear-update-eval-aclosure ac :attribute "default value by type"
      :instance (aget tfs (car tns))))))

```

Затем явно заданные значения заменяют значения по умолчанию.

```

(aclosure ac :attribute "opsem::rvalue" :type "struct lit" :instance i :stage
  "filling defaults"
:ap ac "type field names" tns :ap ac "struct value" sv :value dv
:p (mo "pointer type" :av "type" (mo "pointer type"
  :av "type" (aget i "type" "fields" (car tns)))) ct :do
  (aset sv (car tns) (mo "cell" :av "value" dv :av "type" ct))
  (match :v (null tns) T :do sv

```

```

:exit (update-push-aclosure ac :av "type field names" (cdr tns)
      :av "struct value" sv)
      (clear-update-eval-aclosure ac :attribute "default value by type"
      :instance (aget (aget i "type" "fields") (car (cdr tns)))))

```

Финальное значение структуры создаётся после обработки всех полей.

```

(aclosure ac :attribute "opsem::rvalue" :type "struct lit" :instance i :stage
  "build value"
:value sv :do (mo "struct value" :av "type" (aget i "type") :av "fields"
  sv))

```

3.6. Литерал карты

Вычисление литерала карты происходит путём последовательного вычисления пар ключ-значение.

```

(aclosure ac :attribute "opsem::rvalue" :type "map lit" :instance i :stage nil
  :ap i "elements" es :do
  (update-push-aclosure ac :stage "build value")
  (match :v (null es) T :do nil
    :exit (update-push-aclosure ac :stage "evaluating"
      :av "entries" (mo :amap "Go value" "cell")
      :ap "elements" (cdr es))
    (clear-update-eval-aclosure ac :instance (car es)
      :av "elem type" (aget i "type" "elem type"))))

```

Каждая пара обрабатывается отдельным замыканием.

```

(aclosure ac :attribute "opsem::rvalue" :type "map lit" :instance i :stage
  "evaluating"
:ap ac "entries" ent :ap ac "elements" es :value ke :do
  (aset ent (aget ke "key") (aget ke "value")))

```

```

(match :v (null es) T :do ent
  :exit (update-push-aclosure ac :av "entries" ent :av "elements" (cdr
    es))
    (update-push-aclosure ac :instance (car es)))
(clear-update-eval-aclosure ac :instance (aget (car es) "key")
  :av "elem type" (aget i "type" "elem type")))

```

После вычисления всех пар создаётся итоговая карта.

```

(aclosure ac :attribute "opsem::rvalue" :type "map lit" :instance i :stage
  "build value"
  :value ent :do (mo "map value" :av "type" (aget i "type") :av "entries"
    ent))

```

3.7. Литерал функции

Функциональные литералы создают значения функций, сохраняя текущее состояние замыкания — ячейки памяти всех переменных, видимых в момент создания.

```

(aclosure ac :attribute "opsem::rvalue" :type "function lit" :instance i
  :agent a :p (aget i "body" "all variables") ns :p nil c :do
  (dolist (item ns)
    (setf c (aset c (aget a "variable cell" item)))))
(mo "function value" :av "type" (aget i "type")
  :av "signature" (aget i "signature")
  :av "body" (aget i "body")
  :av "closure" c))

```

3.8. Литерал метода

Аналогично создаётся значение метода.

```

(aclosure ac :attribute "opsem::rvalue" :type "method lit" :instance i

```

```

:agent a :p (aget i "body" "all variables") ns :p nil c :do
(dolist (item ns)
  (setf c (aset c (aget a "variable cell" item))))
(mo "method value" :av "type" (aget i "type")
    :av "signature" (aget i "signature")
    :av "body" (aget i "body")
    :av "closure" c))

```

3.9. Выражения

3.9.1. Ссылка на переменную

Для ссылки на переменную определены два замыкания: для получения левого значения (lvalue) — ячейки памяти, и для получения правого значения (rvalue) — содержимого ячейки.

```

(aclosure ac :attribute "opsem::lvalue" :type "variable ref"
  :instance i :agent a :do (aget a "variable cell" i))
(aclosure ac :attribute "opsem::rvalue" :type "variable ref"
  :instance i :agent a :do (aget (aget a "variable cell" i) "value"))

```

3.9.2. Выражение в скобках

Выражение в скобках вычисляет своё содержимое.

```

(aclosure ac :attribute "opsem::rvalue" :type "(1)" :instance i :do
  (clear-update-eval-aclosure ac :instance (aget i 1)))

```

3.9.3. Приведение типов

Приведение типов вычисляет значение и преобразует его к указанному типу.

```

(aclosure ac :attribute "opsem::rvalue" :type "conversion"
  :instance i :stage nil :do

```



```
(update-push-aclosure ac :stage "conversion")
(clear-update-eval-aclosure ac :instance (aget i "value")))
```

3.9.4. Метод-выражение

Метод-выражение преобразует метод в функцию, где получатель становится первым параметром.

```
(aclosure ac :attribute "opsem::rvalue" :type "method expr" :instance i
  :agent a :p (aget a "method value" (aget i "receiver type")
    (aget i "name"))) mv
:ap mv "type" mt :ap mv "signature" ms :do
(mo "function value"
  :av "type" (mo "function type"
    :av "param types" (cons (aget mt "receiver type")
      (aget mt "param types")))
  :av "variadic type" (aget mt "variadic type")
  :av "result type" (aget mt "result types"))
:av "signature" (mo "function signature"
  :av "parameters" (cons (aget ms "receiver")
    (aget ms "parameters")))
  :av "variadic param" (aget ms "variadic param")
  :av "result" (aget ms "result"))
:av "body" (aget mv "body")
:av "closure" (aget mv "closure")))
```

3.9.5. Выбор поля или метода

Выбор поля или метода обрабатывает два случая: когда слева от точки стоит имя типа (интерфейса) — возвращается метод; когда слева стоит структура или указатель на структуру — возвращается поле.

```
(aclosure ac :attribute "opsem::rvalue" :type "selector expr"
```

```

:instance i :stage nil :ap i "receiver" r :do
  (update-push-aclosure ac :stage "return value")
  (match :v (and (is-instance r "identifier") (aget a "type" r)) T
    :do (aget a "method value" r (aget i "name"))
    :exit (clear-update-eval-aclosure ac :instance (aget i "receiver")))))

```

Для получения левого значения (только для структур).

```

(aclosure ac :attribute "opsem::lvalue" :type "selector expr"
  :instance i :stage nil :do
    (update-push-aclosure ac :stage "return value")
    (clear-update-eval-aclosure ac :instance (aget i "receiver")
      :attribute "opsem::rvalue"))
(aclosure ac :attribute "opsem::lvalue" :type "selector expr"
  :instance i :stage "return value"
  :value r :ap i "name" n :ap r "type" tr :do
    (nmatch :v tr "struct type" :exit (aget r "fields" n)
      :v tr "pointer type" :exit (aget r "target" "value" "fields" n)))

```

3.9.6. Индексирование

Индексирование применимо к массивам, срезам, картам и строкам.

```

(aclosure ac :attribute "opsem::rvalue" :type "index expr"
  :instance i :stage nil :do
    (update-push-aclosure ac :stage "eval index")
    (clear-update-eval-aclosure ac :instance (aget i "indexable")))

```

Возврат значения по индексу.

```

(aclosure ac :attribute "opsem::rvalue" :type "index expr"
  :instance i :stage "return value"
  :value index :ap ac "indexable" indl :ap indl "type" it :do
    (nmatch :v it "array type" :exit (aget (nth index (aget indl "elements"))

```

```

                                "value")
      :v it "slice type" :exit (aget (nth (+ (aget indl "offset") index)
                                (aget indl "array" "elements"))) "value")
      :v it "map type"   :exit (aget (aget indl "entries" index)
                                "value")
      :v it "string type" :exit (char (aget indl "value" index)))

```

3.9.7. Выражение среза

Выражение среза создаёт новый срез из существующего массива, среза или строки.

```

(aclosure ac :attribute "opsem::rvalue" :type "slice expr"
  :instance i :stage nil :do
    (update-push-aclosure ac :stage "eval low")
    (clear-update-eval-aclosure ac :instance (aget i "sequence")))

```

Для массива:

```

(aclosure ac :attribute "opsem::rvalue" :type "slice expr"
  :instance i :stage "array type"
  :ap ac "sequence" s :ap ac "low" low :ap ac "high" high :ap ac "cap" cap
  :ap s "type" st :p (length (aget s "elements")) len :do
    (if (not high) (aset high len))
    (if (not cap) (aset cap (- len low)))
    (mo "slice value"
      :av "type" (mo "slice type" :av "elem type" (aget st "elem type"))
      :av "array" s
      :av "offset" low
      :av "length" (- high low)
      :av "capacity" cap))

```

Для среза (сохраняется ссылка на исходный массив):

```

(aclosure ac :attribute "opsem::rvalue" :type "slice expr"

```

```

:instance i :stage "slice type"
:ap ac "sequence" s :ap ac "low" low :ap ac "high" high :ap ac "cap" cap
:p (length (aget s "array" "type" "len")) len :do
(if (not high) (aset high len))
(if (not cap) (aset cap (- len low)))
(mb "slice value"
  :av "type" (aget s "type")
  :av "array" (aget s "array")
  :av "offset" (+ (aget s "offset") low)
  :av "length" (aget s "length")
  :av "capacity" (aget s "capacity"))

```

Для строки возвращается подстрока (неизменяемая).

```

(aclosure ac :attribute "opsem::rvalue" :type "slice expr"
  :instance i :stage "string type"
  :p (aget ac "sequence" "value") s :ap ac "low" low :ap ac "high" high :do
  (if (not high) (aset high (length s)))
  (subseq s low high))

```

3.10. Унарные выражения

Все унарные выражения сначала вычисляют свой операнд, затем применяют операцию.

```

(aclosure ac :attribute "opsem::rvalue" :type "unary expression" :instance i
  :stage nil :do
  (update-push-aclosure ac :stage "apply")
  (clear-update-eval-aclosure ac :instance (aget i 1)))

```

Вспомогательная функция для приведения целых чисел к нужному типу с учётом переполнения (wrap-around для беззнаковых, знаковое представление для знаковых).

```

(defun convert-int-to-type (tp raw)

```

```

(mo "typed primitive" :av "type" tp :av "value"
  (if (not (is-instance tp "int type")) raw
    (let* ((bs (aget tp "bit size"))
          (v (mod raw (ash 1 bs))))
      (if (or (is-instance tp "unsigned int type")
              (< v (ash 1 (1- bs))))
          v
          (- v (ash 1 (1- bs)))))))

```

Унарные операторы:

```

(aclosure ac :attribute "opsem::rvalue" :type "+1" :stage "apply" :value v
  :do v)
(aclosure ac :attribute "opsem::rvalue" :type "-1" :stage "apply" :value pr
  :ap pr "value" v :ap pr "type" tp :do (convert-int-to-type tp (- v)))
(aclosure ac :attribute "opsem::rvalue" :type "^1" :stage "apply" :value pr
  :ap pr "value" v :ap pr "type" tp :do (convert-int-to-type tp (lognot v)))
(aclosure ac :attribute "opsem::rvalue" :type "!1" :stage "apply" :value v :do
  (mo "typed primitive" :av "type" (co "bool type")
    :av "value" (not (aget v "value"))))

```

Разыменование указателя (может быть как rvalue, так и lvalue).

```

(aclosure ac :attribute "opsem::rvalue" :type "*1" :stage "apply"
  :value tv :ap tv "value" v :do
  (mo "typed primitive" :av "type" (aget tv "type")
    :av "value" (aget v "value")))
(aclosure ac :attribute "opsem::lvalue" :type "*1" :instance i :stage nil :do
  (update-push-aclosure ac :stage "apply")
  (clear-update-eval-aclosure ac :instance (aget i 1)))
(aclosure ac :attribute "opsem::lvalue" :type "*1" :stage "apply" :value v
  :do (aget v "value"))

```

Взятие адреса.

```
(aclosure ac :attribute "opsem::rvalue" :type "&1" :instance i :stage "apply"
  :value tv :ap tv "type" tp :ap tv "value" v :do
  (mo "typed primitive" :av "type" (co "pointer type" :av "type" tp)
    :av "value" (mo "cell" :av "value" v :av "type" tp)))
```

3.11. Бинарные выражения

Бинарные выражения вычисляют левый операнд, затем правый (кроме логических операций с коротким замыканием).

```
(aclosure ac :attribute "opsem::rvalue" :type "binary expression" :instance i
  :stage nil :do
  (update-push-aclosure ac :stage "apply 1")
  (clear-update-eval-aclosure ac :instance (aget i 1)))
```

Логические операции с коротким замыканием.

```
(aclosure ac :attribute "opsem::rvalue" :type "1||2" :instance i :stage
  "apply 1" :value v :do
  (match :v (aget v "value") T
    :do (mo "typed primitive" :av "type" (co "bool type") :av "value" T)
    :exit (update-push-aclosure ac :stage "apply 2")
    (clear-update-eval-aclosure ac :instance (aget i 2))))
(aclosure ac :attribute "opsem::rvalue" :type "1&&2" :instance i :stage
  "apply 1" :value v :do
  (match :v (aget v "value") nil
    :do (mo "typed primitive" :av "type" (co "bool type") :av "value" nil)
    :exit (update-push-aclosure ac :stage "apply 2")
    (clear-update-eval-aclosure ac :instance (aget i 2))))
```

Арифметические и битовые операции. Результат приводится к типу левого операнда.

```
(aclosure ac :attribute "opsem::rvalue" :type "1+2" :instance i :stage "apply
  1" :value v :do
  (update-push-aclosure ac :stage "apply 2" :av 1 v)
  (clear-update-eval-aclosure ac :instance (aget i 2)))
(aclosure ac :attribute "opsem::rvalue" :type "1+2" :stage "apply 2"
 :ap ac 1 f :ap f "type" tp :ap f "value" fv :value s :ap s "value" sv :do
 (convert-int-to-type tp (+ fv sv)))
```

```
;; subtraction
```

```
(aclosure ac :attribute "opsem::rvalue" :type "1-2" :instance i :stage "apply
  1" :value v :do
  (update-push-aclosure ac :stage "apply 2" :av 1 v)
  (clear-update-eval-aclosure ac :instance (aget i 2)))
(aclosure ac :attribute "opsem::rvalue" :type "1-2" :stage "apply 2"
 :ap ac 1 f :ap f "type" tp :ap f "value" fv :value s :ap s "value" sv :do
 (convert-int-to-type tp (- fv sv)))
```

```
;; multiplication
```

```
(aclosure ac :attribute "opsem::rvalue" :type "1*2" :instance i :stage "apply
  1" :value v :do
  (update-push-aclosure ac :stage "apply 2" :av 1 v)
  (clear-update-eval-aclosure ac :instance (aget i 2)))
(aclosure ac :attribute "opsem::rvalue" :type "1*2" :stage "apply 2"
 :ap ac 1 f :ap f "type" tp :ap f "value" fv :value s :ap s "value" sv :do
 (convert-int-to-type tp (* fv sv)))
```

```
;; division
```

```
(aclosure ac :attribute "opsem::rvalue" :type "1/2" :instance i :stage "apply
  1" :value v :do
  (update-push-aclosure ac :stage "apply 2" :av 1 v)
  (clear-update-eval-aclosure ac :instance (aget i 2)))
(aclosure ac :attribute "opsem::rvalue" :type "1/2" :stage "apply 2"
```

```

      :ap ac 1 f :ap f "type" tp :ap f "value" fv :value s :ap s "value" sv :do
      (convert-int-to-type tp (/ fv sv)))

;; remainder
(aclosure ac :attribute "opsem::rvalue" :type "1%2" :instance i :stage "apply
  1" :value v :do
  (update-push-aclosure ac :stage "apply 2" :av 1 v)
  (clear-update-eval-aclosure ac :instance (aget i 2)))
(aclosure ac :attribute "opsem::rvalue" :type "1%2" :stage "apply 2"
  :ap ac 1 f :ap f "type" tp :ap f "value" fv :value s :ap s "value" sv :do
  (convert-int-to-type tp (rem fv sv)))

;; left shift
(aclosure ac :attribute "opsem::rvalue" :type "1<<2" :instance i :stage
  "apply 1" :value v :do
  (update-push-aclosure ac :stage "apply 2" :av 1 v)
  (clear-update-eval-aclosure ac :instance (aget i 2)))
(aclosure ac :attribute "opsem::rvalue" :type "1<<2" :stage "apply 2"
  :ap ac 1 f :ap f "type" tp :ap f "value" fv :value s :ap s "value" sv :do
  (convert-int-to-type tp (ash fv sv)))

;; right shift
(aclosure ac :attribute "opsem::rvalue" :type "1>>2" :instance i :stage
  "apply 1" :value v :do
  (update-push-aclosure ac :stage "apply 2" :av 1 v)
  (clear-update-eval-aclosure ac :instance (aget i 2)))
(aclosure ac :attribute "opsem::rvalue" :type "1>>2" :stage "apply 2"
  :ap ac 1 f :ap f "type" tp :ap f "value" fv :value s :ap s "value" sv :do
  (convert-int-to-type tp (ash fv (- sv))))

;; bitwise AND
(aclosure ac :attribute "opsem::rvalue" :type "1&2" :instance i :stage "apply

```



```

1" :value v :do
(update-push-aclosure ac :stage "apply 2" :av 1 v)
(clear-update-eval-aclosure ac :instance (aget i 2)))
(aclosure ac :attribute "opsem::rvalue" :type "1&2" :stage "apply 2"
:ap ac 1 f :ap f "type" tp :ap f "value" fv :value s :ap s "value" sv :do
(convert-int-to-type tp (logand fv sv)))

;; bitwise AND NOT
(aclosure ac :attribute "opsem::rvalue" :type "1&^2" :instance i :stage
"apply 1" :value v :do
(update-push-aclosure ac :stage "apply 2" :av 1 v)
(clear-update-eval-aclosure ac :instance (aget i 2)))
(aclosure ac :attribute "opsem::rvalue" :type "1&^2" :stage "apply 2"
:ap ac 1 f :ap f "type" tp :ap f "value" fv :value s :ap s "value" sv :do
(convert-int-to-type tp (logand fv (lognot sv))))

;; bitwise OR
(aclosure ac :attribute "opsem::rvalue" :type "1|2" :instance i :stage "apply
1" :value v :do
(update-push-aclosure ac :stage "apply 2" :av 1 v)
(clear-update-eval-aclosure ac :instance (aget i 2)))
(aclosure ac :attribute "opsem::rvalue" :type "1|2" :stage "apply 2"
:ap ac 1 f :ap f "type" tp :ap f "value" fv :value s :ap s "value" sv :do
(convert-int-to-type tp (logior fv sv)))

;; bitwise XOR
(aclosure ac :attribute "opsem::rvalue" :type "1^2" :instance i :stage "apply
1" :value v :do
(update-push-aclosure ac :stage "apply 2" :av 1 v)
(clear-update-eval-aclosure ac :instance (aget i 2)))
(aclosure ac :attribute "opsem::rvalue" :type "1^2" :stage "apply 2"
:ap ac 1 f :ap f "type" tp :ap f "value" fv :value s :ap s "value" sv :do

```

```
(convert-int-to-type tp (logxor fv sv)))
```

Операции сравнения возвращают типизированное булево значение.

```
(aclosure ac :attribute "opsem::rvalue" :type "1==2" :instance i :stage
  "apply 1" :value v :do
    (update-push-aclosure ac :stage "apply 2" :av 1 v)
    (clear-update-eval-aclosure ac :instance (aget i 2)))
(aclosure ac :attribute "opsem::rvalue" :type "1==2" :stage "apply 2"
  :ap ac 1 f :ap f "type" tp :ap f "value" fv :value s :ap s "value" sv :do
    (convert-int-to-type tp (= fv sv)))
```

```
;; inequality
(aclosure ac :attribute "opsem::rvalue" :type "1!=2" :instance i :stage
  "apply 1" :value v :do
    (update-push-aclosure ac :stage "apply 2" :av 1 v)
    (clear-update-eval-aclosure ac :instance (aget i 2)))
(aclosure ac :attribute "opsem::rvalue" :type "1!=2" :stage "apply 2"
  :ap ac 1 f :ap f "type" tp :ap f "value" fv :value s :ap s "value" sv :do
    (convert-int-to-type tp (not (= fv sv))))
```

```
;; less than
(aclosure ac :attribute "opsem::rvalue" :type "1<2" :instance i :stage "apply
  1" :value v :do
    (update-push-aclosure ac :stage "apply 2" :av 1 v)
    (clear-update-eval-aclosure ac :instance (aget i 2)))
(aclosure ac :attribute "opsem::rvalue" :type "1<2" :stage "apply 2"
  :ap ac 1 f :ap f "type" tp :ap f "value" fv :value s :ap s "value" sv :do
    (convert-int-to-type tp (< fv sv)))
```

```
;; less than or equal
(aclosure ac :attribute "opsem::rvalue" :type "1<=2" :instance i :stage
  "apply 1" :value v :do
```

```

(update-push-aclosure ac :stage "apply 2" :av 1 v)
(clear-update-eval-aclosure ac :instance (aget i 2)))
(aclosure ac :attribute "opsem::rvalue" :type "1<=2" :stage "apply 2"
  :ap ac 1 f :ap f "type" tp :ap f "value" fv :value s :ap s "value" sv :do
  (convert-int-to-type tp (<= fv sv)))

;; greater than
(aclosure ac :attribute "opsem::rvalue" :type "1>2" :instance i :stage "apply
  1" :value v :do
  (update-push-aclosure ac :stage "apply 2" :av 1 v)
  (clear-update-eval-aclosure ac :instance (aget i 2)))
(aclosure ac :attribute "opsem::rvalue" :type "1>2" :stage "apply 2"
  :ap ac 1 f :ap f "type" tp :ap f "value" fv :value s :ap s "value" sv :do
  (convert-int-to-type tp (> fv sv)))

;; greater than or equal
(aclosure ac :attribute "opsem::rvalue" :type "1>=2" :instance i :stage
  "apply 1" :value v :do
  (update-push-aclosure ac :stage "apply 2" :av 1 v)
  (clear-update-eval-aclosure ac :instance (aget i 2)))
(aclosure ac :attribute "opsem::rvalue" :type "1>=2" :stage "apply 2"
  :ap ac 1 f :ap f "type" tp :ap f "value" fv :value s :ap s "value" sv :do
  (convert-int-to-type tp (>= fv sv)))

```

3.12. Декларации

3.12.1. Декларация типа

Декларация типа добавляет определение типа в окружение агента.

```

(aclosure ac :attribute "opsem" :type "type decl" :instance i :agent a :do
  (aset a "type" (aget i "name") (aget i "type")))

```

3.12.2. Декларация переменных

Декларация переменных может происходить двумя способами: с явным указанием значений или без них (используются значения по умолчанию).

```
(aclosure ac :attribute "opsem" :type "var decl" :instance i :stage nil
  :ap ac "block" b :ap i "names" ns :ap i "values" vs :ap i "types" ts :do
    (match :v (null vs) T
      :do (update-push-aclosure ac :stage "declarating defaults"
        :av "names" ns :av "types" (cdr ts) :av "block" b)
        (clear-update-eval-aclosure ac :attribute "default value by type"
          :instance (car ts))
        :exit (update-push-aclosure ac :stage "evaluating values"
          :av "names" ns :av "types" ts :av "values" (cdr vs) :av
            "block" b)
          (clear-update-eval-aclosure ac :instance (car vs))))))
```

3.12.3. Декларация функций и методов

Декларация функции вычисляет значение функционального литерала и сохраняет его в окружении.

```
(aclosure ac :attribute "opsem" :type "function decl"
  :instance i :stage nil :do
    (update-push-aclosure ac :stage "decl")
    (clear-update-eval-aclosure ac :instance (aget i "value")))
(aclosure ac :attribute "opsem" :type "function decl"
  :instance i :stage "decl" :value v :agent a :do
    (aset a "function value" (aget i "name") v))
```

3.13. Блоки

Блоки определяют области видимости переменных. При входе в блок сохраняются старые ячейки всех переменных, объявленных в этом блоке.

```
(aclosure ac :attribute "opsem" :type "block" :instance i :stage nil
  :ap i "statements" sts :ap i "decl variables" avs :p nil dv :agent a :do
  (aset i "variable cells" (dolist (item avs (reverse dv))
    (setf dv (cons (aget a "variable cell" item) dv))))
  (update-push-aclosure ac :stage "variable back")
  (clear-update-eval-aclosure ac :stage "evaluating statement" :av
    "statements" sts))
```

Вычисление очередного оператора блока.

```
(aclosure ac :attribute "opsem" :type "block"
  :instance i :stage "evaluating statement"
  :ap ac "statements" sts :v (null sts) nil :do
  (update-push-aclosure ac :av "statements" (cdr sts))
  (clear-update-eval-aclosure ac :instance (car sts)))
```

Восстановление старых ячеек при выходе из блока.

```
(aclosure ac :attribute "opsem" :type "block"
  :instance i :stage "variable back" :do
  (dolist (item (aget i "decl variables"))
    (aset a "variable cell" item (aget item "variable cells"))))
```

3.14. Операторы присваивания

Присваивание сначала вычисляет левую часть (lvalue), затем правую (rvalue).

```
(aclosure ac :attribute "opsem" :type "1=2" :instance i :stage nil :do
  (update-push-aclosure ac :stage "eval 2")
  (clear-update-eval-aclosure ac :instance (aget i 2)))
(aclosure ac :attribute "opsem" :type "1=2" :instance i :stage "eval 2"
```

```

:value v :do
(update-push-aclosure ac :stage "apply" :ac 1 v)
(clear-update-eval-aclosure ac :instance (aget i 1) :attribute
"opsem::lvalue"))
(aclosure ac :attribute "opsem" :type "1=2" :stage "apply" :ap ac 1 f :value
s :do
(aset f "value" (aget s "value"))))

```

Составные присваивания (например, `+=`) выполняют операцию перед записью.

```

(aclosure ac :attribute "opsem" :type "1+=2" :stage "apply" :ap ac 1 f :value
s :do
(aset f "value" (convert-int-to-type (aget f "type")
(+ (aget f "value") (aget s "value")))))

```

3.15. Условные операторы

Условный оператор `if`.

```

(aclosure ac :attribute "opsem" :type "if stmt" :instance i :stage nil :do
(update-push-aclosure ac :stage "apply" :av "block" (aget ac "block"))
(clear-update-eval-aclosure ac :instance (aget i "condition")))
(aclosure ac :attribute "opsem" :type "if stmt" :stage "apply" :value c :ap
ac "block" b :do
(clear-update-eval-aclosure ac :instance (aget i (if c "then" "else"))
:av "block" b))

```

3.16. Циклы

Цикл `for condition`.

```

(aclosure ac :attribute "opsem" :type "for condition" :instance i :stage nil
:ap ac "block" b :do

```

```

(update-push-aclosure ac :stage "apply" :av "block" b)
(clear-update-eval-aclosure ac :instance (aget i "condition"))
(aclosure ac :attribute "opsem" :type "for condition" :stage "apply"
:value v :v v T :av ac "block" b :do
(update-push-aclosure ac :stage nil :av "block" b)
(clear-update-eval-aclosure ac :instance (aget i "body"))))

```

3.17. Операторы передачи управления

Оператор `return`.

```

(aclosure ac :attribute "opsem" :type "return stmt" :instance i :do
(clear-update-eval-aclosure ac :instance (aget i 1)))
(aclosure ac :attribute "opsem" :type "return stmt" :stage "ret"
:value v :do (aset a "value" (car v)))

```

3.18. Канальные операции

Оператор отправки в канал.

```

(aclosure ac :attribute "opsem" :type "1<-2" :instance i :do
(update-push-aclosure ac :stage "send-value")
(clear-update-eval-aclosure ac :instance (aget i 1)))
(aclosure ac :attribute "opsem" :type "1<-2" :instance i :stage "send-value"
:value ch :ap i 2 e2 :do
(update-push-aclosure ac :stage "send" :av "channel" ch)
(clear-update-eval-aclosure ac :instance e2))
(aclosure ac :attribute "opsem" :type "1<-2" :stage "send"
:ap ac "channel" ch :value val :do
(aset ch "buffer" (append (aget ch "buffer") (list (aget val "value"))))))

```

Операция чтения из канала как выражение.

```
(aclosure ac :attribute "opsem::rvalue" :type "<-1" :instance i :do
  (clear-update-eval-aclosure ac :instance (aget i 1)))
(aclosure ac :attribute "opsem::rvalue" :type "<-1" :stage "receive"
  :value ch :do (pop (aget ch "buffer")))
```